Experiment 26: Gibbs Sampling and Contrastive Divergence Analysis

Question

Problem Statement: The streaming team finds that its RBM learns slowly and poorly. The research lead asks how many Gibbs steps are worth the extra computation. The binarised MNIST or user–movie data from Experiment 25. The objective is to analyse Gibbs sampling chains and compare CD-1, CD-k and Persistent CD. Use NumPy, SciPy, Matplotlib.

Tasks:

1. Implement block Gibbs sampling between visible and hidden layers.
2. Run chains from random noise and display samples after 1, 10, 100 and 1000 steps.
3. Train RBMs with CD-1, CD-5, CD-10 and PCD.
4. Plot reconstruction error and pseudo-log-likelihood for each method.
5. Compare training time against sample quality.

Code

```
"""Experiment 26: Gibbs Sampling and Contrastive Divergence Analysis (CD-1, CD-5, CD-10, PCD)"""
import time
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit

rng = np.random.RandomState(0)

def load_mnist(n_train=5000, n_test=500):
    try:
        from tensorflow import keras
        (x, _), (xt, _) = keras.datasets.mnist.load_data()
        x = x.reshape(-1, 784) / 255.0; xt = xt.reshape(-1, 784) / 255.0
    except Exception as e:
        print("WARNING: MNIST not available (", type(e).__name__, ") -> using scikit-learn digits scaled to
28x28.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        d = load_digits().images / 16.0
        d = np.stack([zoom(im, 3.5, order=1) for im in d]).reshape(-1, 784)
        x, xt = d[:int(0.8 * len(d))], d[int(0.8 * len(d)):]
    return (x[:n_train] > 0.5).astype(float), (xt[:n_test] > 0.5).astype(float)

X_train, X_test = load_mnist()
print("Train:", X_train.shape, "Test:", X_test.shape)

class RBM:
    def __init__(self, nv=784, nh=64):
        self.W = rng.normal(0, 0.01, (nv, nh)); self.a = np.zeros(nv); self.b = np.zeros(nh)
        self.chain = None                                          # persistent chain for PCD
    def p_h(self, v): return expit(v @ self.W + self.b)
    def p_v(self, h): return expit(h @ self.W.T + self.a)
    def sample(self, p): return (rng.rand(*p.shape) < p).astype(float)
    def free_energy(self, v): return -(v @ self.a) - np.sum(np.logaddexp(0, v @ self.W + self.b), axis=1)

    # ---- Task 1: block Gibbs sampling between the visible and hidden layers ----
    def gibbs(self, v, steps):
        for _ in range(steps):
            h = self.sample(self.p_h(v))
            v = self.sample(self.p_v(h))
        return v

    def train_batch(self, v0, lr, k, persistent=False):
        ph0 = self.p_h(v0)
        if persistent:                                             # PCD: continue the chain from the last
update
            if self.chain is None:
                self.chain = v0.copy()
```

```
            vk = self.gibbs(self.chain, k); self.chain = vk
        else:                                                   # CD-k: start the chain at the data
            vk = self.gibbs(v0, k)
        phk = self.p_h(vk)
        B = len(v0)
        self.W += lr * (v0.T @ ph0 - vk.T @ phk) / B
        self.a += lr * np.mean(v0 - vk, axis=0)
        self.b += lr * np.mean(ph0 - phk, axis=0)


def recon_error(rbm, V):
    return np.mean((V - rbm.p_v(rbm.p_h(V))) ** 2)

def pseudo_log_likelihood(rbm, V):                              # stochastic PLL estimate (flip one
random bit per sample)
    idx = rng.randint(0, V.shape[1], len(V)); Vf = V.copy()
    Vf[np.arange(len(V)), idx] = 1 - Vf[np.arange(len(V)), idx]
    return V.shape[1] * np.mean(np.log(expit(rbm.free_energy(Vf) - rbm.free_energy(V)) + 1e-12))

# ---------------- Task 3 & 4: train CD-1, CD-5, CD-10, PCD ----------------
EPOCHS, BATCH, LR = 8, 100, 0.05
methods = {"CD-1": (1, False), "CD-5": (5, False), "CD-10": (10, False), "PCD": (1, True)}
results, models = {}, {}
for name, (k, pers) in methods.items():
    rbm = RBM(); rec, pll = [], []
    t0 = time.time()
    for ep in range(EPOCHS):
        perm = rng.permutation(len(X_train))
        for i in range(0, len(X_train) - BATCH + 1, BATCH):       # full batches only (PCD chain has fixed
size)
            rbm.train_batch(X_train[perm[i:i + BATCH]], LR, k, pers)
        rec.append(recon_error(rbm, X_test)); pll.append(pseudo_log_likelihood(rbm, X_test))
    results[name] = {"recon": rec, "pll": pll, "time": time.time() - t0}
    models[name] = rbm
    print(f"{name:6} time={results[name]['time']:6.1f}s  recon MSE={rec[-1]:.4f}  PLL={pll[-1]:8.2f}")

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
for name, r in results.items():
    ax[0].plot(range(1, EPOCHS + 1), r["recon"], "o-", label=name); ax[1].plot(range(1, EPOCHS + 1),
r["pll"], "o-", label=name)
ax[0].set_title("Test reconstruction error (MSE)"); ax[1].set_title("Pseudo-log-likelihood (higher is
better)")
for a in ax:
    a.set_xlabel("Epoch"); a.legend(); a.grid(alpha=0.3)
plt.tight_layout(); plt.show()

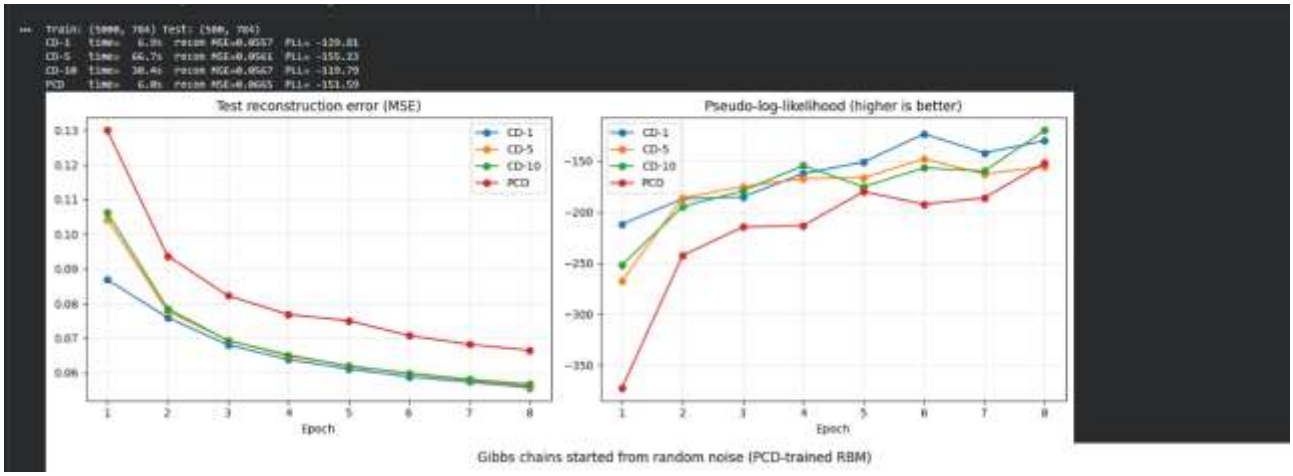
# ---------------- Task 2: Gibbs chains from random noise on a trained RBM ----------------
rbm = models["PCD"]
checkpoints = [1, 10, 100, 1000]
n_chains = 8
v = (rng.rand(n_chains, 784) < 0.5).astype(float)
snapshots, done = {}, 0
for cp in checkpoints:
    v = rbm.gibbs(v, cp - done); done = cp
    snapshots[cp] = rbm.p_v(rbm.sample(rbm.p_h(v)))                # probabilities shown for a cleaner
picture
fig, axes = plt.subplots(len(checkpoints), n_chains, figsize=(2 * n_chains, 2.1 * len(checkpoints)))
for r, cp in enumerate(checkpoints):
    for c in range(n_chains):
        axes[r, c].imshow(snapshots[cp][c].reshape(28, 28), cmap="gray"); axes[r, c].axis("off")
    axes[r, 0].set_title(f"After {cp} Gibbs step(s)", loc="left", fontsize=10)
plt.suptitle("Gibbs chains started from random noise (PCD-trained RBM)"); plt.tight_layout(); plt.show()

# ---------------- Task 5: training time vs sample quality ----------------
fig, axes = plt.subplots(len(methods), 8, figsize=(14, 7))
for r, (name, m) in enumerate(models.items()):
    s = rng.rand(8, 784) < 0.5
    s = m.gibbs(s.astype(float), 500)
    for c in range(8):
        axes[r, c].imshow(m.p_v(m.sample(m.p_h(s)))[c].reshape(28, 28), cmap="gray"); axes[r, c].axis("off")
    axes[r, 0].set_title(f"{name}: samples after 500 Gibbs steps", loc="left", fontsize=9)
plt.tight_layout(); plt.show()

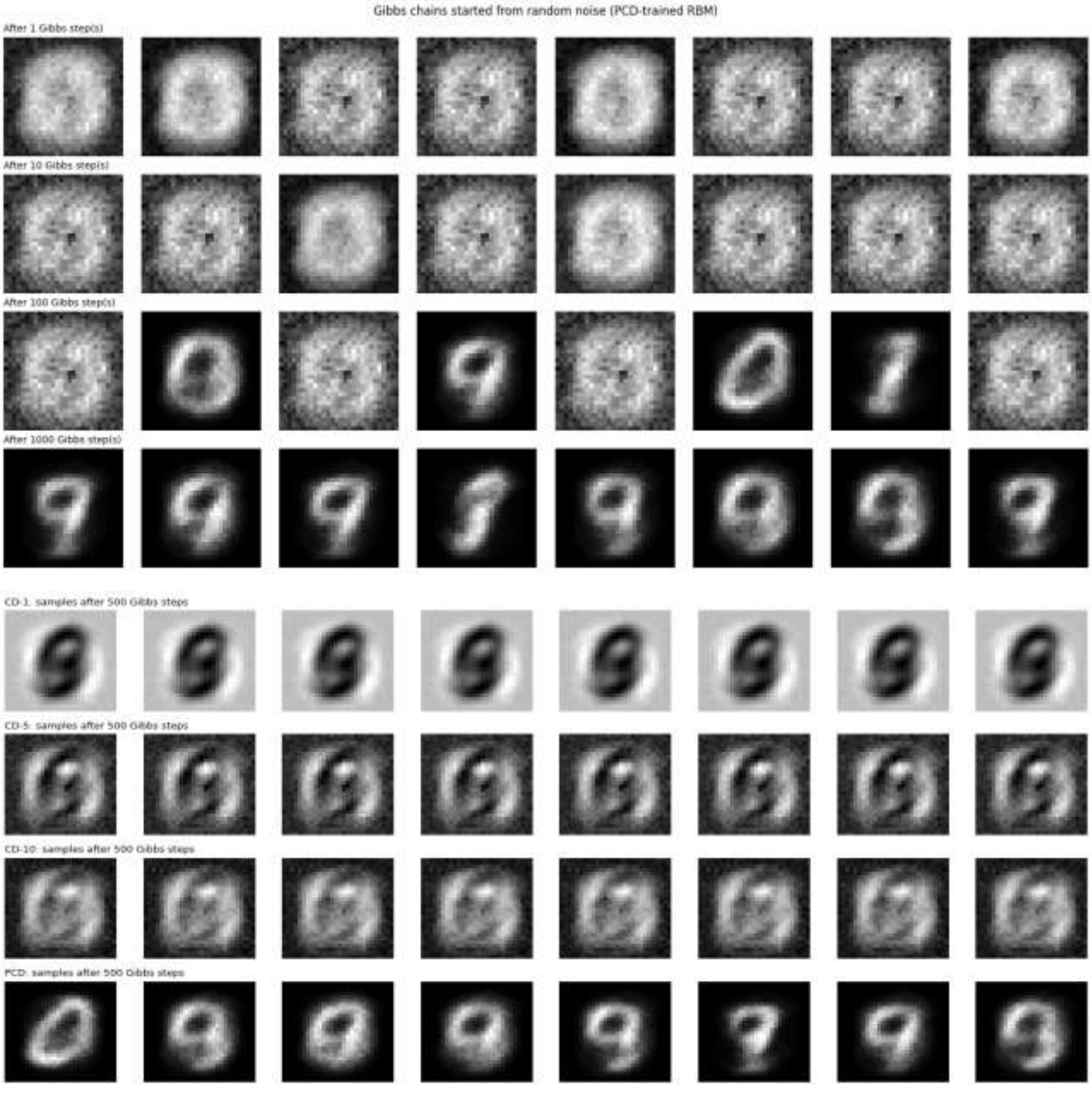
print(f"\n{'Method':8}{'Time (s)':>10}{'Recon MSE':>12}{'PLL':>10}")
for name, r in results.items():
```

```
    print(f"{name:8}{r['time']:10.1f}{r['recon'][-1]:12.4f}{r['pll'][-1]:10.2f}")
print("\nSummary: CD-k cost grows roughly in proportion to k, while PCD costs about the same as CD-1 because its "
      "chain keeps running between updates. Compare the PLL and sample quality columns to decide whether the "
      "extra Gibbs steps were worth the time.")
```

Output



Gibbs chains started from random noise (PCD-trained RBM)





Method	Time (s)	Mean MSE	PSL
CD-1	6.9	0.0557	−125.91
CD-4	68.7	0.0555	−191.22
CD-10	38.4	0.0557	−115.73
PCE	5.8	0.0655	−151.59

Summary: CD-4 cost grows roughly in proportion to k, while PCE costs about the same as CD-1 because its chain keeps running between updates. Compare the PSL and sample quality columns to decide whether the extra CD4ks steps were worth the time.